Lecture # 5

By: Muhammad Zain Uddin

email: zuddin@iba.edu.pk

Quick Review

Multithreading

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
typedef struct {
    int start;
    int end;
    int* data;
    int* partial_sum;
} thread_data_t;

void* worker(void* arg) {
    thread_data_t* data = (thread_data_t*)arg;
    int sum = 0;
    for (int i = data->start; i < data->end; ++i) {
        sum += data->data[i] * data->data[i];
    }
    int* result = (int*)malloc(sizeof(int));
    *result = sum;
    return result;
}
int sum_of_squares_multithread(int data[], int len, int num_threads)
{
    pthread_t threads[num_threads];
    thread_data_t thread_data[num_threads];
```

```
for (int i = 0; i < num_threads; ++i) {
        thread_data[i].start = i * (len / num_threads);
        thread_data[i].end = (i == num_threads - 1) ? len : (i + 1) * (len / num_threads);
        thread_data[i].data = data;
        thread_data[i].partial_sum = NULL;
        pthread_create(&threads[i], NULL, worker, &thread_data[i]);
    }

    int total_sum = 0;
    for (int i = 0; i < num_threads; ++i) {
        void* result;
        pthread_join(threads[i], &result);
        total_sum += *(int*)result;
        free(result);
    }

    return total_sum;
}

int main() {
    int data[] = {2, 5, 7, 4};
    int sum = sum_of_squares_multithread(data, 4, 2);
    printf("Sum of squares: %d\n", sum);

    return 0;
}
```

Headers:
stdio.h: Provides functions for input/output operations like printing to the console.
pthread.h: Contains functions and types for working with threads, enabling multithreading.
stdlib.h: Offers general utility functions, including memory allocation and deallocation.

Structure:
thread_data_t: A structure for packaging information to be passed to worker threads:
Start: The starting index of the data segment assigned to the thread.
end: The ending index of the data segment.
data: A pointer to the shared array containing the data to be processed.

Worker Function:
worker:
Takes a thread_data_t pointer as input, providing thread-specific data.
Calculates the sum of squares within its assigned data segment.
Dynamically allocates memory using malloc to store the calculated partial sum.
Stores the partial sum in the allocated memory location.
Returns a pointer to the newly allocated memory containing the partial sum.

```
void* worker(void* arg) {
    thread_data_t* data = (thread_data_t*)arg;
    int sum = 0;
    for (int i = data->start; i < data->end; ++i) {
        sum += data->data[i] * data->data[i];
    }
    int* result = (int*)malloc(sizeof(int));
    *result = sum;
    return result;
}
```

Sum of Squares Function:

sum_of_squares_multithread:

Accepts the data array, its length, and the desired number of threads as input.

Creates an array of threads and thread_data_t structures to organize thread-related information.

Divide the data array into chunks and assign them to different threads for parallel processing.

Initializes a variable total_sum to 0, which will accumulate the final result.

Creates the threads using pthread_create, each executing the worker function with its corresponding thread_data_t.

Waits for all threads to finish using pthread_join, collecting their partial sums.

Iterates through the returned partial sums, adding them to total_sum.

Frees the memory allocated for partial sums using free to prevent memory leaks.

Returns the calculated total_sum, representing the sum of squares of all elements in the array.

```
int sum_of_squares_multithread(int data[], int len, int num_threads) {
    pthread_t threads[num_threads];
    thread_data_t thread_data[num_threads];

for (int i = 0; i < num_threads; ++i) {
        thread_data[i].start = i * (len / num_threads);
        thread_data[i].end = (i == num_threads - 1) ? len : (i + 1) * (len / num_threads);
        thread_data[i].data = data;
        thread_data[i].partial_sum = NULL;
        pthread_create(&threads[i], NULL, worker, &thread_data[i]);
    }

    int total_sum = 0;
    for (int i = 0; i < num_threads; ++i) {
        void* result;
        pthread_join(threads[i], &result);
        total_sum += *(int*)result;
        free(result);
    }

    return total_sum;
}
```

Main Function:

main:

Creates a sample data array to be processed.

Calls the sum_of_squares_multithread function to perform the multithreaded calculation.

Prints the final sum of squares obtained from the multithreaded operation.

Returns 0 to indicate successful

Multithreading(2)

```
#include <stdio.h>
#include <pthread.h>

typedef struct {
  int start;
  int end;
  int* data;
  int* partial_sum;
} thread_data_t;

int total_sum = 0;
pthread_mutex_t mutex;

void* worker(void* arg) {
  thread_data_t* data = (thread_data_t*)arg;
  int partial_sum = 0;
  for (int i = data->start; i < data->end; ++i) {
    partial_sum += data->data[i] * data->data[i];
  }

  pthread_mutex_lock(&mutex);  // Protect shared variable access
  total_sum += partial_sum;
  pthread_mutex_unlock(&mutex);

  return NULL;
}
int sum_of_squares_multithread(int data[], int len, int num_threads)
{
  pthread_t threads[num_threads];
  thread_data_t thread_data[num_threads];
```

```
  for (int i = 0; i < num_threads; ++i) {
    thread_data[i].start = i * (len / num_threads);
    thread_data[i].end = (i == num_threads - 1) ? len : (i + 1) * (len / num_threads);
    thread_data[i].data = data;
    thread_data[i].partial_sum = NULL;  // Not used in this version
    pthread_create(&threads[i], NULL, worker, &thread_data[i]);
  }

  pthread_mutex_init(&mutex, NULL);  // Initialize mutex

  for (int i = 0; i < num_threads; ++i) {
    pthread_join(threads[i], NULL);  // Wait for threads to finish
  }

  pthread_mutex_destroy(&mutex);  // Clean up mutex

  return total_sum;
}

int main() {
  int data[] = {2, 5, 7, 4};
  int sum = sum_of_squares_multithread(data, 4, 2);
  printf("Sum of squares: %d\n", sum);

  return 0;
}
```

Headers:
stdio.h: Provides standard input/output functions like printf.
pthread.h: Includes functions for multithreading, such as thread creation and mutexes.
Structure:

thread_data_t: Represents data passed to worker threads:
start: Starting index for the thread's data segment.
end: Ending index for the thread's data segment.
data: Pointer to the shared array of integers.
partial_sum: Not used in this version.
Global Variables:

total_sum: Stores the cumulative sum of squares, shared among threads.
mutex: Mutex lock to protect total_sum from concurrent access.
Worker Function:

worker:
Takes a thread_data_t pointer as input.
Calculates a partial sum of squares within its assigned data segment.
Acquires the mutex lock to protect total_sum.
Adds the partial sum to total_sum.
Releases the mutex lock.
Returns NULL.

```
void* worker(void* arg) {
  thread_data_t* data = (thread_data_t*)arg;
  int partial_sum = 0;
  for (int i = data->start; i < data->end; ++i) {
    partial_sum += data->data[i] * data->data[i];
  }

  pthread_mutex_lock(&mutex);  // Protect shared variable access
  total_sum += partial_sum;
  pthread_mutex_unlock(&mutex);

  return NULL;
}
```

Sum of Squares Function:

sum_of_squares_multithread:
Takes the data array, its length, and the desired number of threads as input.
Creates an array of threads and thread_data_t structures.
Divides the data into chunks and assigns them to threads.
Initializes the mutex lock.
Creates the threads, passing worker and corresponding thread_data_t.
Waits for all threads to finish using pthread_join.
Destroys the mutex lock.
Returns the final sum of squares (total_sum).
Main Function:

main:
Creates a sample data array.
Calls sum_of_squares_multithread with the array, its length, and 2 threads.
Prints the calculated sum of squares.
Returns 0 to indicate successful execution.

```
int sum_of_squares_multithread(int data[], int len, int num_threads) {
  pthread_t threads[num_threads];
  thread_data_t thread_data[num_threads];

  for (int i = 0; i < num_threads; ++i) {
    thread_data[i].start = i * (len / num_threads);
    thread_data[i].end = (i == num_threads - 1) ? len : (i + 1) * (len / num_threads);
    thread_data[i].data = data;
    thread_data[i].partial_sum = NULL;  // Not used in this version
    pthread_create(&threads[i], NULL, worker, &thread_data[i]);
  }

  pthread_mutex_init(&mutex, NULL);  // Initialize mutex

  for (int i = 0; i < num_threads; ++i) {
    pthread_join(threads[i], NULL);  // Wait for threads to finish
  }

  pthread_mutex_destroy(&mutex);  // Clean up mutex

  return total_sum;
}
```

Multithreading(3)

g++ -o myprogram TH_EXAMP3.C -pthread -lstdc++

```
#include <stdio.h>
#include <pthread.h>
#include <time.h>
#include <mutex>

typedef struct {
  int start;
  int end;
  int* data;
  int* partial_sum;
} thread_data_t;

std::mutex mtx;  // Global mutex for synchronization

void* worker(void* arg) {
  thread_data_t* data = (thread_data_t*)arg;
  int sum = 0;
  for (int i = data->start; i < data->end; ++i) {
    sum += data->data[i] * data->data[i];
  }

  std::lock_guard<std::mutex> lock(mtx);  // Acquire mutex
before updating partial sum
  *(data->partial_sum) += sum;

  return NULL;
}
```

```
int sum_of_squares_multithread(int  data[], int len, int num_threads) {
  pthread_t threads[num_threads];
  thread_data_t thread_data[num_threads];
  int total_sum = 0;
  int chunk_size = len / num_threads;

for (int i = 0; i < num_threads; ++i) {
    thread_data[i].start = i * chunk_size;
    thread_data[i].end = (i == num_threads - 1) ? len : (i + 1) * chunk_size;
    thread_data[i].data = data;
    thread_data[i].partial_sum = new int(0);  // Separate partial sum for each thread
    pthread_create(&threads[i], NULL, worker, &thread_data[i]);
  }
  for (int i = 0; i < num_threads; ++i) {
    pthread_join(threads[i], NULL);
    std::lock_guard<std::mutex> lock(mtx);
    total_sum += *(thread_data[i].partial_sum);  // Aggregate partial sums safely
    delete thread_data[i].partial_sum;
  }
  return total_sum;
}
int main() {
  int data[] = {2, 5, 7, 4};

  clock_t start_time = clock();
  int sum = sum_of_squares_multithread(data,  4, 2);
  clock_t end_time = clock();

  printf("Sum of squares: %d\n", sum);
  printf("Execution Time: %f seconds\n", ((double)(end_time - start_time)) /
CLOCKS_PER_SEC);

  return 0;
}
```

Structure:

Data Structures:
thread_data_t: This structure holds information for each thread, including its start and end indices within the data array, a pointer to the data itself, and a pointer to an integer partial_sum.
global mutex: A single std::mutex object used for thread synchronization when accessing the total sum.

Functions:
worker: This function is executed by each thread. It calculates the sum of squares for its assigned subset of the data array and stores it in the partial_sum variable. Then, it acquires the global mutex lock, adds its partial sum to the total sum, releases the lock, and exits.
sum_of_squares_multithread: This function performs the overall multithreaded computation. It creates threads, defines and assigns data to each thread, joins the threads to wait for completion, and finally aggregates the partial sums into the final total sum.

Algorithm:

- **Initialization:**
  - Define the data array, number of threads, and chunk size for dividing the work.
  - Allocate and initialize thread data structures and the global mutex.

- **Thread Creation:**
  - For each thread, set its start and end indices based on the chunk size and thread ID.
  - Create a thread and pass the corresponding thread_data_t structure as an argument.

- **Thread Execution:**
  - Each thread iterates over its assigned sub-array, calculating the sum of squares for each element.
  - The thread then acquires the global mutex lock, ensuring atomic access to the total sum.
  - It adds its partial sum to the total sum and releases the lock.

- **Result Aggregation:**
  - The main thread waits for all created threads to finish using pthread_join.
  - After all threads are done, it iterates through the partial_sum variables of each thread data structure.
  - It acquires the mutex lock again for each iteration to protect the total sum.
  - It adds each partial sum to the total sum and releases the lock.

- **Cleanup:**
  - The main thread frees the allocated memory for each thread data structure.
  - It destroys the global mutex.

Difference of three code

Functionality:
All three codes calculate the sum of squares of elements in an array using multiple threads.
Structure:
Code 1: Simplest structure, but requires dynamic memory allocation for partial_sum in each thread.
Code 2: Global mutex for synchronization, potentially introduces contention.
Code 3: Separate partial_sum for each thread, reducing mutex usage and contention.
Synchronization:
Code 1: Lacks synchronization on total_sum, leading to potential race conditions and incorrect results.
Code 2: Uses a global mutex to protect total_sum, avoids race conditions.
Code 3: Minimal mutex usage, only acquires lock when aggregating partial sums.
Performance:
Code 1: Potentially faster due to lack of synchronization, but unreliable.
Code 2: May suffer from mutex contention, impacting performance with many threads.
Code 3: Generally better performance and scalability due to reduced contention.

Difference of three codes (con.t)

Memory Usage:

Code 1: Requires dynamic memory allocation for partial_sum in each thread, potentially impacting memory overhead.

Code 2: No additional memory usage beyond data and thread structures.

Code 3: Similar to Code 2, but adds the cost of separate partial_sum variables.

Recommendation:

Code 3: Generally the best choice for most multithreaded scenarios due to its combination of:

Thread safety through synchronization.

Improved performance and scalability with minimal contention.

Reasonable memory usage.

Code 2: Acceptable if contention is not a major concern and simplicity is preferred.

Code 1: Avoidable in multithreaded contexts as it lacks proper synchronization and can lead to inaccurate results.

Additional Considerations:

False Sharing: All three codes could be susceptible to false sharing depending on memory layout and hardware. consider strategies like padding or data reorganization to mitigate this.

Overhead: Thread creation and synchronization have inherent overhead. Consider if the gains from multithreading outweigh this overhead for small datasets or tasks.

Alternative Parallelization: Explore libraries like OpenMP or TBB for simplified parallel programming and potential further performance optimizations.

Posix Threading API:

All three codes use the Posix Threading API for creating and managing threads. This API provides functions like pthread_create, pthread_join, and pthread_mutex_init/destroy to handle thread creation, synchronization, and resource management.

- pthread_create: Spawns a new thread to run a function with arguments.
- pthread_join: Waits for a specific thread to finish its execution.
- pthread_mutex_init/destroy: Initializes and cleans up a mutex object for thread synchronization.
- pthread_mutex_lock/unlock: Acquires and releases a lock on a mutex, managing access to shared resources.

Code 1 and 2 use mutexes (pthread_mutex_t) to synchronize access to the shared total_sum variable and avoid race conditions.

Code 3 introduces separate partial_sum variables for each thread, reducing the need for frequent mutex access and potential contention.

GDB (GNU Debugger)

1. Compile with debugging symbols:
Run this command in your terminal to compile the code with debugging information:
gcc TH_EXAMP.C -o THREAD
2. Start GDB:
Open a new terminal window and type the following command to start GDB:
gdb sum_squares
3. Set breakpoints:
Use the break command followed by the line number or function name to set breakpoints at the places you want to pause the program execution. Here's how you can set breakpoints at the commented lines in your code:
•Observe thread creation: break main
•Inspect final result: break main:14 (replace 14 with the actual line number where you want to break after printing the sum)
•Observe worker thread execution: break worker
•Observe thread synchronization: break sum_of_squares_multithread
4. Run the program:
After setting your breakpoints, type run to start the program execution.
5. Use GDB commands at breakpoints:
Once the program hits a breakpoint, you can use various GDB commands to inspect the state of your program. Here are some helpful commands:
•info threads: Display information about all threads, including their IDs and states.
•thread N: Switch to thread N (replace N with the thread ID).
•print var: Print the value of a variable named var.
•next: Step over the next line of code without entering a function.
•step: Step into the next line of code, even if it's a function call.
•continue: Resume program execution until the next breakpoint or the end of the program.

vector processing

vector_examp.c

```
#include <immintrin.h>

#include <stdio.h>

#include <time.h>

__m128 sum_of_squares_vector(float data[], int len) {

  __m128 total = _mm_setzero_ps();  // Initialize vector accumulator (using __m128 for floats)

  int chunks = len / 4;

  for (int i = 0; i < chunks; ++i) {

    __m128 chunk = _mm_loadu_ps(data + i * 4);  // Load 4 floats into a vector register

    // Consider aligned load (_mm_load_ps) if data is aligned

    chunk = _mm_mul_ps(chunk, chunk);  // Square each element using vector instructions

    total = _mm_add_ps(total, chunk);  // Accumulate the squared elements}

  return total;  // Return the vector containing the sum of squares

}}
```

gcc -msse3 -o vector_examp

```
int main() {
  float data[] = {2.0f, 5.0f, 7.0f, 4.0f};  // Use floats for SIMD compatibility
  clock_t start_time = clock();
  __m128 sum = sum_of_squares_vector(data, 4);

  // Extract sum of all elements using horizontal addition:
  __m128 sum_horizontal = _mm_hadd_ps(sum, sum);  // Combine elements pairwise
  sum_horizontal = _mm_hadd_ps(sum_horizontal, sum_horizontal); // Combine pairs again
  float total_sum = _mm_cvtss_f32(sum_horizontal);   // Extract final sum
  clock_t end_time = clock();

  printf("Sum of squares: %f\n", total_sum);
  printf("Execution Time: %f seconds\n", ((double)(end_time - start_time)) / CLOCKS_PER_SEC);

  return 0;
}
```

1. Header Inclusion:

immintrin.h: Provides intrinsic functions for accessing SIMD instructions (like _mm_setzero_ps, _mm_loadu_ps, _mm_mul_ps, _mm_add_ps, _mm_hadd_ps, and _mm_cvtss_f32).
stdio.h: Contains functions for input/output, like printf.

2. Function for Sum of Squares Using SIMD:

__m128 Data Type: Represents a 128-bit vector register capable of holding 4 single-precision floating-point values.
_mm_setzero_ps(): Initializes a vector register to all zeros.
_mm_loadu_ps(data + i * 4): Loads 4 floats from memory into a vector register, allowing for unaligned data.
_mm_mul_ps(chunk, chunk): Performs element-wise multiplication of a vector with itself, squaring each element.
_mm_add_ps(total, chunk): Adds two vectors element-wise, accumulating the squared values.

3. Main Function:

_mm_hadd_ps(sum, sum): Performs horizontal addition, combining elements pairwise to reduce the number of active elements.
_mm_cvtss_f32(sum_horizontal): Extracts a single float from a vector register.
printf("Sum of squares: %f\n", total_sum): Prints the calculated sum of squares.

Advantages and Disadvantages of all

Array Processing:
Functionality: Applies the same operation to all elements of an array simultaneously.
Advantages: Faster than SISD due to avoiding loop overhead, utilizes CPU caches efficiently.
Disadvantages: Still sequential data access, not as efficient for highly parallelizable tasks.

Vector Processing:
Functionality: Exploits SIMD instructions to operate on multiple data elements within a single vector register simultaneously.
Advantages: Significantly faster than array processing for vectorizable tasks, utilizes dedicated vector units on CPUs.
Disadvantages: Requires specific data types and instruction sets, more complex code, limited efficiency for non-vectorizable tasks.

Multi-thread Processing:
Functionality: Splits the workload into smaller chunks and distributes them among multiple threads for parallel execution.
Advantages: Exploits multiple CPU cores to potentially achieve significant speedups, suitable for independent tasks.
Disadvantages: Overhead of thread creation and management, synchronization challenges, may not always provide benefits depending on task and hardware.

Synchronization

- **Definition:**
  - Synchronization involves coordinating the execution of threads or processes to maintain consistency and order.
- **Types of Synchronization:**
  - **Semaphores:** Control access to resources with a counter.
  - **Mutex (Mutual Exclusion):**
    - Protect critical sections by allowing only one thread at a time.

Mutual Exclusion

Definition:

- Mutual exclusion ensures that only one thread or process accesses a critical section at a time.

Example:

Scenario:

- Two threads attempting to update a shared counter simultaneously.

Techniques for Mutual Exclusion

- **Mutex (Mutual Exclusion):**
  - A synchronization primitive to protect shared resources.
  - Example
    - pthread_mutex_t mutex;
    - pthread_mutex_init(&mutex, NULL);
    - pthread_mutex_lock(&mutex);
    - // Critical section
    - pthread_mutex_unlock(&mutex);

- **Semaphore:**
  - A synchronization primitive for controlling access to a shared resource
  - Example
    - sem_t semaphore;
    - sem_init(&semaphore, 0, 1); // Initialize with a counter of 1
    - sem_wait(&semaphore);
    - // Critical section
    - sem_post(&semaphore);

Feature	Mutex	Semaphore
Mechanism-used	Mutex uses a locking mechanism.	Semaphore uses a signaling mechanism.
Data-type	A mutex is an object.	A semaphore is an integer variable.
Modification:	Mutex can only be modified by the process that either requests or releases a resource.	Wait and signal operations can modify a semaphore.
Number of threads	Multiple program threads in mutex can exist but not simultaneously.	Multiple threads can exist.
Operation	Wait() and signal() operations modify the operation semaphore value.	Mutex Object is either locked or unlocked.
Resource management	If no resource is free then the wait operation is executed by the process.	If it is the mutex is locked then the process must wait and should be kept in a queue.
Types	There are two kinds of Semaphore, counting semaphore and binary semaphore.	No subtypes exist.

MUTEX

```
#include <stdio.h>
#include <pthread.h>
int shared_counter = 0;
pthread_mutex_t mutex;
void* update_counter(void* arg) {
    pthread_mutex_lock(&mutex);  // Acquire the lock to ensure mutual exclusion
    shared_counter++;
    printf("Thread %ld updated counter to %d\n", (long)arg, shared_counter);
    pthread_mutex_unlock(&mutex);  // Release the lock after updating the counter
    return NULL;
}
int main() {
    pthread_t thread1, thread2;
    pthread_mutex_init(&mutex, NULL);  // Initialize the mutex

    // Create two threads to update the shared counter
    pthread_create(&thread1, NULL, update_counter, (void*)1);
    pthread_create(&thread2, NULL, update_counter, (void*)2);
    // Wait for both threads to complete
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    pthread_mutex_destroy(&mutex);  // Destroy the mutex
    return 0;
}
```

SEMAPHORE

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
int shared_counter = 0;
sem_t semaphore;
void* update_counter(void* arg) {
   sem_wait(&semaphore);  // Wait on the semaphore, decrementing its value
   shared_counter++;
   printf("Thread %ld updated counter to %d\n", (long)arg, shared_counter);
   sem_post(&semaphore);  // Signal the semaphore, incrementing its value
   return NULL;
}
int main() {
   pthread_t thread1, thread2;
   sem_init(&semaphore, 0, 1);  // Initialize the semaphore with an initial value of 1
   // Create two threads to update the shared counter
   pthread_create(&thread1, NULL, update_counter, (void*)1);
   pthread_create(&thread2, NULL, update_counter, (void*)2);
   // Wait for both threads to complete
   pthread_join(thread1, NULL);
   pthread_join(thread2, NULL);
   sem_destroy(&semaphore);  // Destroy the semaphore after its use
   return 0;}
```

OpenMP

- OpenMP is an extension of C used for multi-threaded code (shared memory, so no multi-node computation)
- Compiled with the additional flag "gcc -fopenmp foo.c"
  - "#include <omp.h>"
- Standardized over many languages
- Generally follows the fork-join framework
- Each thread has its own stack for private variables, but otherwise shares memory with other threads
- OpenMP code generally is written using lines like "#pragma omp <command>"

#pragma omp parallel

- In order to create a parallel section:

```
#pragma omp parallel
{
  //parallel code
}
```

- C syntax note: brackets mean "previous declaration applies to the all the lines inside me". This means that if we write only one line of code in a parallel section (or if clause, or for loop), we don't need to write brackets.
- The code in the parallel section gets run on all threads, so we need some way to distinguish threads
- In a parallel segment:
    - `omp_get_num_threads()` returns the number of threads running
    - `opm_get_thread_num()` returns a unique number from 0-num_threads per thread

Parallel Hello World

```
#include <stdio.h>
#include <omp.h>
int main () {
    int x = 0; //Shared variable
      #pragma omp parallel
      {
            int tid = omp_get_thread_num(); //Private variable
            x++;
            printf("Hello World from thread %d, x = %d\n", tid, x);
            if(tid==0) {
                  printf("Number of threads = %d\n", omp_get_num_threads());
            }
      }
      printf("Done with parallel segment\n");
}
```

Shared vs Private variables

- By default, any variable declared outside the parallel segment is shared: all threads write/read from the same variable
- Any variable declared inside the parallel segment is private: Each thread has its own version of the variable.
- Can overrule this with the private and shared keywords:

```
#pragma omp parallel private(var) shared(var2)
```

- Note that heap memory is always shared, though we can have private pointers and private mallocs (if we do the malloc in the parallel segment, and free them within the parallel segment)

For loops

- Problem: You have to do some work over an array of 1 million numbers, with 4 people. How do you split the work?
- Assumptions:
  - We need to decide this before we run the code
  - Each element of the array is independent, so we can do this in any order
  - The threads are about equally fast at work, so we want to assign each of them 250k numbers

For loops

- Option 1: Do every 4
  - `for(int i = tid; i<1000000;i+=4)`
  - "Interweaving"
- Option 2: Thread 0 does 0-249999, 1 does 250000-499999, etc.
  - `for(int i = tid*250000;i<(tid+1)*250000;i++)`
  - "Blocking"
- Which one's better?
  - With standard multithreading, Option 1 actually is as slow as the serial version of this code due to cache coherency issues… More on this in coming class
  - Option 2 speeds things up correctly
  - Aside: Option 1 does end up being the preferred option when dealing with GPU programming, though that's due to how GPU threads differ from normal threads.

For loops

- Option 3: Let the compiler do it for you with the `#pragma omp for` keyword.
- `#pragma omp for` must be written inside an already existing parallel segment
- If a parallel segment consists only of one for loop, we can combine the two declarations with `#pragma omp parallel for`
  -
```
#pragma omp parallel for
    for(int i = 0; i<1000000;i++)
```

Data Races

- Note that when we ran Hello World parallel, we ended up with the threads running in random order
  - In fact, every time we run Hello World, we get a different order!
  - The x values stayed largely in-order, but didn't always strictly increase
- Recall the OS can choose whichever threads it wants to run, and change threads at any time
- This is one of the biggest downsides to multithreading: A multithreaded program is no longer deterministic, and will have a random execution order every time we run the program.
- Formally, a multithreaded program is only considered correct if ANY interlacing of threads yield the same result.

Data Race: Example

- To analyze this, we need to see the equivalent assembly code
  - C will compile to x86, but we can still do a correct analysis by compiling to RISC-V, since we're mainly trying to reduce the code to atomic instructions. We can assume that no two atomic instructions happen simultaneously.
- Only the loads and stores affect shared memory, so we only need to consider the different ways we can order the loads and stores
- Even with this, there are $8!/(2!)^4=2520$ different possible orders
  - Can use the fact that all the threads are identical to reduce this to 105 orders, but still too many to check manually

```
sw x0 0(sp)
```


lw t0 0(sp)	lw t0 0(sp)	lw t0 0(sp)	lw t0 0(sp)
addi t0 t0 1	addi t0 t0 1	addi t0 t0 1	addi t0 t0 1
sw t0 0(sp)	sw t0 0(sp)	sw t0 0(sp)	sw t0 0(sp)

Data Race: Example

- Case 1: All the threads run one at a time
  - Purple thread reads x = 0
  - Purple thread stores x = 1
  - Brown thread reads x = 1
  - Brown thread stores x = 2
  - Red thread reads x = 2
  - Red thread stores x = 3
  - Blue thread reads x = 3
  - Blue thread stores x = 4
- Final value: 4

```
sw x0 0(sp)
lw t0 0(sp)
addi t0 t0 1
sw t0 0(sp)
lw t0 0(sp)
addi t0 t0 1
sw t0 0(sp)
lw t0 0(sp)
addi t0 t0 1
sw t0 0(sp)
lw t0 0(sp)
addi t0 t0 1
sw t0 0(sp)
```

Data Race: Example

- Case 2: The threads are perfectly interleaved
  - Purple thread reads x = 0
  - Red thread reads x = 0
  - Brown thread reads x = 0
  - Blue thread reads x = 0
  - Purple thread stores x = 1
  - Brown thread stores x = 1
  - Red thread stores x = 1
  - Blue thread stores x = 1
- Final value: 1

```
sw x0 0(sp)
lw t0 0(sp)
lw t0 0(sp)
lw t0 0(sp)
lw t0 0(sp)
addi t0 t0 1
addi t0 t0 1
addi t0 t0 1
addi t0 t0 1
sw t0 0(sp)
sw t0 0(sp)
sw t0 0(sp)
sw t0 0(sp)
```

Data Race: Example

- Case 3: Same as case 1, except purple's store happens last
  - Purple thread reads x = 0
  - Brown thread reads x = 0
  - Brown thread stores x = 1
  - Red thread reads x = 1
  - Red thread stores x = 2
  - Blue thread reads x = 2
  - Blue thread stores x = 3
  - Purple thread stores x = 1
- Final value: 1

```
sw x0 0(sp)
lw t0 0(sp)
addi t0 t0 1
lw t0 0(sp)
addi t0 t0 1
sw t0 0(sp)
lw t0 0(sp)
addi t0 t0 1
sw t0 0(sp)
lw t0 0(sp)
addi t0 t0 1
sw t0 0(sp)
sw t0 0(sp)
```

Data Race: Example

- Some other ordering?
  - We can find orderings that give x=2,3
- Can we do any more/less?
- Can't go above 4
  - Only 4 "+1s" overall, so can't increase to 5 or more
- Can't go below 1
  - The smallest value that can be loaded by a thread is 0, so the smallest value that can be stored is 1. Therefore, the last store must be at least 1.
- Therefore, we can get any value between 1 and 4

```
sw x0 0(sp)
lw t0 0(sp)
lw t0 0(sp)
lw t0 0(sp)
addi t0 t0 1
addi t0 t0 1
addi t0 t0 1
sw t0 0(sp)
lw t0 0(sp)
addi t0 t0 1
sw t0 0(sp)
sw t0 0(sp)
sw t0 0(sp)
```

```
sw x0 0(sp)
lw t0 0(sp)
lw t0 0(sp)
addi t0 t0 1
addi t0 t0 1
sw t0 0(sp)
lw t0 0(sp)
addi t0 t0 1
sw t0 0(sp)
lw t0 0(sp)
addi t0 t0 1
sw t0 0(sp)
sw t0 0(sp)
```

Avoiding Data Races

- Formally, a multithreaded program is only considered correct if ANY interlacing of threads yield the same result.
- For today: if you make sure that each thread works on independent data (no two threads write to the same value, or read a value that another thread wrote to), you can guarantee correctness
- Tommorow: how to handle cases where coordination is mandatory
- The hardest part of multithreading is maintaining correctness while also speeding up the code, but this ends up being a fairly transferable skill to project management
  - If you can coordinate a group of threads to perform a task, you can coordinate a group of people to perform a task more easily.

Data Races/Race Conditions

- Note that when we ran Hello World parallel, we ended up with the threads running in random order
  - In fact, every time we run Hello World, we get a different order!
  - The x values stayed largely in-order, but didn't always strictly increase
- Recall the OS can choose whichever threads it wants to run, and change threads at any time
- This is one of the biggest downsides to multithreading: A multithreaded program is no longer deterministic, and will have a random execution order every time we run the program.
- Formally, a multithreaded program is only considered correct if ANY interlacing of threads yield the same result.

Data Race Example: Retrospective

- In practice, most times you run this code, you get 4
  - Each thread is small enough that it's unlikely to get interrupted
- Empirical tests suggest an error rate around 0.01%.
- In summation, race condition bugs are:
  - Rare
  - Nondeterministic
  - Silent-failing (you get the wrong answer instead of crashing the program)
- Very difficult to debug. You have been warned…

Avoiding Data Races

- Formally, a multithreaded program is only considered correct if ANY interlacing of threads yield the same result.
- If you make sure that each thread works on independent data (no two threads write to the same value, or read a value that another thread wrote to), you can guarantee correctness
- The hardest part of multithreading is maintaining correctness while also speeding up the code, but this ends up being a fairly transferable skill to management
  - If you can coordinate a group of threads to perform a task, you can coordinate a group of people to perform a task more easily.
- How to handle cases where coordination is mandatory?

Locks

- A lock is an object which helps with synchronization
- Essentially, each thread can try to "acquire" a lock, but only one thread can have the lock at a given time.
  - Think bathroom stall lock; only one person can use the stall at a time, and we use atomics to make sure two people don't go into the same stall at the same time
- Formally, has two operations
  - acquire: Tries to acquire the lock. If successful, keep going. Otherwise, wait a bit and try again later.
  - release: Unlocks the lock and continues. Only works if we had the lock to start with.
  - Optional but common: try-acquire: Same as acquire, but if the lock is being used, return false and let the thread continue running
- Code surrounded by a lock is called a "critical section", because only one thread is allowed to run that section at a time.

Locks: Downsides

- Using a lock inherently means you need to pause one thread while waiting for another thread to run the critical segment
- Ends up making some parts of your code serial
  - Amdahl's Law strikes again!
- Is it possible for all threads to get stuck?